

732A94 Advanced R programming

Lecture 4

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzysztof Bartoszek)

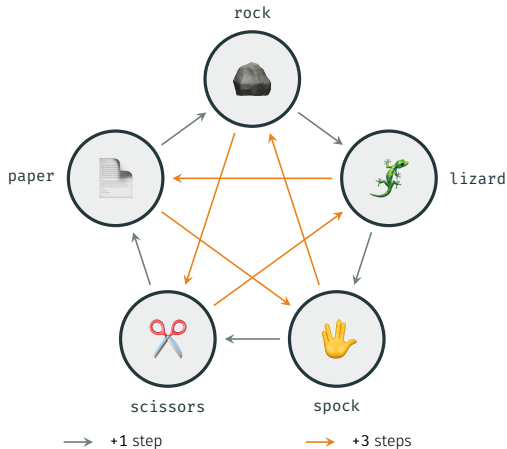
Linköpings Universitet

- Linear algebra in R
- Dynamic documentation with knitr and R Markdown
- ggplot2
- Object orientation
 - S3
 - S4
 - Reference classes

Questions?

Any questions from last time?

Rock-paper-scissors-lizard-spock



An arrow points from the winner to the loser: each alternative beats the two that are 1 and 3 steps ahead in the cycle.

sheldon_game (idea, not full solution)

```
sheldon_game <- function(player1, player2) {  
  alt <- c("rock", "lizard", "spock", "scissors", "paper")  
  stopifnot(player1 %in% alt, player2 %in% alt)  
  alt1 <- which(alt %in% player1)  
  alt2 <- which(alt %in% player2)  
  if (any((alt1 + c(1, 3)) %% 5 == alt2)) {  
    return("Player 1 wins!")  
  } else {  
    return("Player 2 wins!")  
  }  
}
```

Does it work?

```
sheldon_game("rock", "scissors")
```

```
## [1] "Player 1 wins!"
```

```
sheldon_game("scissors", "paper")
```

```
## [1] "Player 2 wins!"
```

Does it work?

```
sheldon_game("rock", "scissors")
```

```
## [1] "Player 1 wins!"
```

```
sheldon_game("scissors", "paper")
```

```
## [1] "Player 2 wins!"
```

Scissors *cuts* paper, so that second answer is wrong.

`alt1 = 4` gives `(4 + c(1, 3)) %% 5 = 0, 2` – but R indexes from 1, so **0** should have been 5.

Also: what about a draw?

Basics are in **base**

- uses LINPACK or LAPACK under the hood

Extra functionality: the **Matrix** package

- extra LAPACK functionality, and sparse matrices

Symbolic algebra: e.g. the **caracas** package

Linear algebra

```
A <- matrix(1:9, ncol = 3)  # create matrix
A <- A + diag(3)             # make it invertible
b <- c(1, 2, 3)
t(A)                         # transpose
```

```
##      [,1] [,2] [,3]
## [1,]    2    2    3
## [2,]    4    6    6
## [3,]    7    8   10
```

```
A %*% A                      # matrix multiplication
```

```
##      [,1] [,2] [,3]
## [1,]   33   74  116
## [2,]   40   92  142
## [3,]   48  108  169
```

```
solve(A)           # matrix inversion
```

```
##      [,1] [,2] [,3]  
## [1,]   -6 -1.0    5  
## [2,]   -2  0.5    1  
## [3,]    3  0.0   -2
```

```
solve(A, b)        # solve  $Ax = b$ 
```

```
## [1]  7  2 -3
```

```
eigen(A)$values    # eigenvalues
```

```
## [1] 17.116844  1.000000 -0.116844
```

```
det(A)             # determinant
```

```
## [1] -2
```

```
qr(A)$rank         # QR factorization
```

```
## [1] 3
```

Also available: `svd(A)`, `chol(A)` (Cholesky, for positive definite matrices),
`cbind(A, A)` / `rbind(A, A)` for block matrices.

Do not invert if you can solve

```
set.seed(1)
n <- 400
X <- matrix(rnorm(n * n), n); y <- rnorm(n)
system.time(replicate(20, solve(X) %*% y))["elapsed"]
```

```
## elapsed
##      0.057
```

```
system.time(replicate(20, solve(X, y)))["elapsed"]
```

```
## elapsed
##      0.029
```

`solve(X, y)` is faster *and* numerically more stable than forming the inverse. The same holds for `lm.fit()` versus normal equations.

```
x <- matrix(rnorm(10000 * 1000), 10000, 1000)
system.time(t(x) %*% x)[ "elapsed"]
```

```
## elapsed
##    0.094
```

```
system.time(crossprod(x))[ "elapsed"]
```

```
## elapsed
##    0.038
```

`crossprod(x)` computes $X^T X$ without ever forming X^T , and uses a BLAS routine that exploits the symmetry of the result – so it does about half the work. Similarly `tcrossprod(x)` gives XX^T .

“Let us change our traditional attitude to the construction of programs: Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to humans what we want the computer to do.”

- Good documentation is essential for code to remain relevant
- Programs are sometimes so complex that comments in the source code do not adequately describe the author's thoughts – program maintenance and reproducible research require detailed documentation
- Several practical use cases require well-formatted code documentation
- **Dynamic documentation** allows code to be embedded within the documentation, so the document explains what the code does *and* runs it

- The main workhorse of dynamic documentation containing R code is the **knitr** package
- **knitr** supports embedding code in various languages (R, Python, SQL, C++, ...) in various document formats (markdown, LaTeX, HTML)
- **pandoc** converts markdown documents into a number of different formats – PDF, Word, HTML, ...
- R Markdown combines **knitr** and **pandoc**
- RStudio provides a convenient interface for developing, previewing and knitting R Markdown files

`.Rmd` $\longrightarrow$ `knitr` $\longrightarrow$ `.md` $\longrightarrow$ `pandoc` $\longrightarrow$ `.pdf` / `.html` / `.docx`

1. You write the `.Rmd` file
2. `knitr` runs every code chunk and replaces it with its output
3. That leaves a plain markdown file
4. `pandoc` converts markdown into the final format of your choice

Components: metadata

- Metadata are optional, but when present they are at the beginning of the .Rmd file
- Demarcated with three dashes ---
- Written in YAML

```
---  
title: "Untitled"  
author: "John Doe"  
date: '2026-09-14'  
output:  
  pdf_document: default  
  html_document:  
    df_print: paged  
---
```

- The narrative is written in Markdown
- Inline formatting: **bold**, *italics*, ~subscript~, ^superscript^, ``code``
- Header levels: # level 1, ## level 2, ...
- Math through LaTeX syntax between dollar signs: $\alpha_0 = 10^2$
gives $\alpha_0 = 10^2$
- References as in LaTeX, with a .bib file and [@refid] in the text

See <https://bookdown.org/yihui/rmarkdown/markdown-syntax.html>

Components: code

- Code is written in **chunks** demarcated by three backticks
- The opening backticks must be followed by the language in curly braces, e.g. `{r}`
- Optional chunk options follow, separated by commas
- Inline code is written between single backticks

```
```{r, echo=FALSE}  
x <- rnorm(100)
```
```

The sample mean is ``r round(mean(x), 2)``.

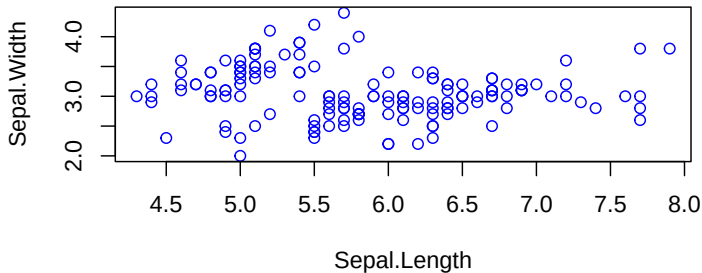
Chunk options worth knowing

| Option | Effect |
|------------------------------------|--|
| <code>echo</code> | Show the source code |
| <code>eval</code> | Actually run the code |
| <code>include</code> | Show anything at all (code <i>and</i> output) |
| <code>error</code> | TRUE keeps knitting when the code fails |
| <code>message, warning</code> | Show messages/warnings |
| <code>fig.width, fig.height</code> | Figure size in inches |
| <code>out.width</code> | Size in the output document |
| <code>cache</code> | Re-use results if the chunk is unchanged |

Set defaults for the whole document with `knitr::opts_chunk$set()`.

A simple example

```
```{r, fig.height=2}  
plot(iris$Sepal.Length, iris$Sepal.Width, col = "blue")
```
```



Everything you see was produced by `Lecture4.Rmd`:

```
output:
  beamer_presentation:
    slide_level: 2
    keep_tex: true
    latex_engine: "lualatex"
```

`slide_level: 2` means every `##` header starts a new slide. `\pause` still works, and `keep_tex: true` leaves the generated `.tex` behind so you can see what pandoc did.

A popular visualization package.

“The grammar of graphics” – the language of visualization.

Flexible: you build the plot from components rather than calling one big function.

Cheat sheet: <https://rstudio.github.io/cheatsheets/data-visualization.pdf>

Create a graph layer by layer.

Store it as an object (printing the object draws the plot).

Three main parts:

data the dataset with the observations to be visualized (a `data.frame`)

geom the geometric representation of the data

aes the mapping from variables to position, colour, shape, ...

| geom | Use |
|------------------------------|---------------------------------------|
| <code>geom_histogram</code> | Histograms (continuous univariate) |
| <code>geom_density</code> | Density plots (continuous univariate) |
| <code>geom_bar</code> | Bar chart (discrete univariate) |
| <code>geom_point</code> | Scatterplots (continuous bivariate) |
| <code>geom_boxplot</code> | Boxplot (discrete-continuous) |
| <code>geom_density_2d</code> | 2D density (continuous bivariate) |
| <code>geom_contour</code> | Contour plots (trivariate) |
| <code>geom_line</code> | Lineplots (versatile) |

| aesthetic | Meaning |
|-----------|--|
| x | X-axis variable; name of a column in data |
| y | Optional; Y-axis variable |
| size | Optional; width of line/point |
| colour | Optional; colour corresponding to the geom |
| shape | Optional; shape corresponding to the geom |
| fill | Optional; fill colour |
| group | Optional; which rows belong to the same object |

See <https://ggplot2.tidyverse.org/articles/ggplot2-specs.html>

| geom | Special aesthetics |
|------------|--------------------------|
| geom_point | shape, size |
| geom_line | linetype, linewidth |
| geom_bar | ymin, ymax, fill, colour |

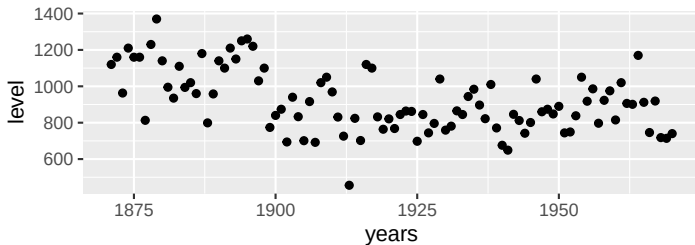
An aesthetic set *inside* `aes()` maps a variable; set *outside* `aes()` it is a constant, e.g. `geom_point(colour = "blue")`.

```
library(ggplot2)
data(Nile)
Nile <- data.frame(level = as.numeric(Nile))
Nile$years <- 1871:1970
Nile$period <- "- 1900"
Nile$period[Nile$years >= 1900] <- "1900 - 1945"
Nile$period[Nile$years > 1945] <- "1945 +"
Nile$period <- as.factor(Nile$period)
head(Nile, 3)
```

```
##   level years period
## 1  1120  1871 - 1900
## 2  1160  1872 - 1900
## 3   963  1873 - 1900
```

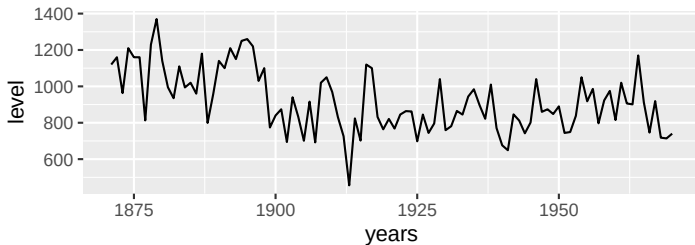
ggplot2: geom_point

```
pl <- ggplot(data = Nile) +  
  aes(x = years, y = level) +  
  geom_point()  
pl
```



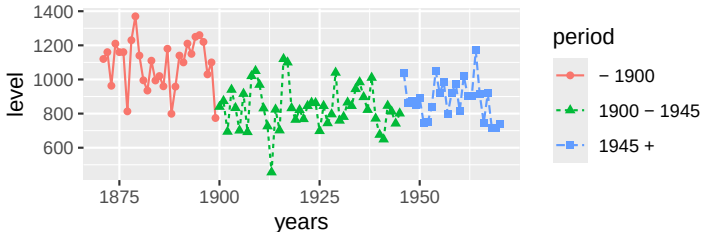
ggplot2: geom_line

```
pl <- ggplot(data = Nile) +  
  aes(x = years, y = level) +  
  geom_line()  
pl
```



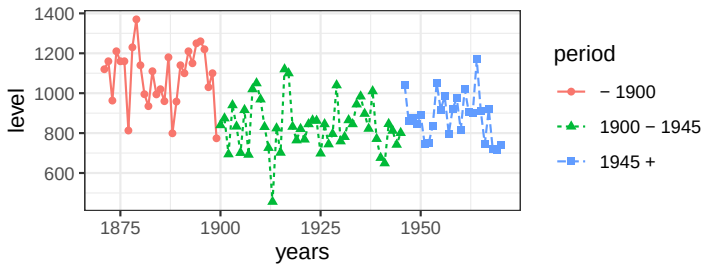
ggplot2: points + lines + colours

```
pl <- ggplot(data = Nile) +  
  aes(x = years, y = level, color = period) +  
  geom_line(aes(linetype = period)) +  
  geom_point(aes(shape = period))  
pl
```



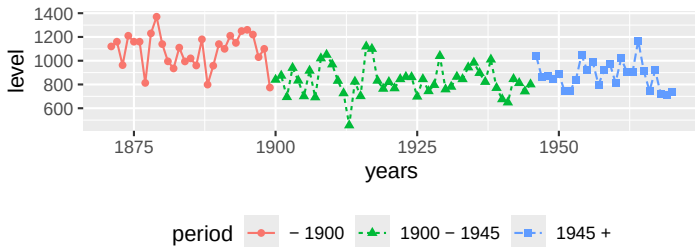
ggplot2: use the BW theme

```
pl + theme_bw()
```

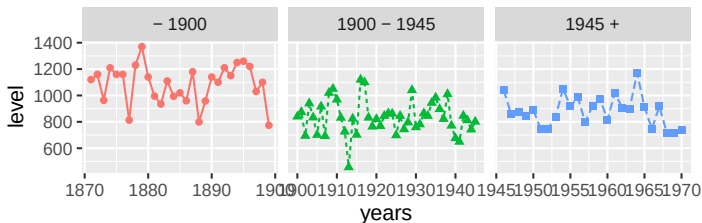


ggplot2: change the legend position

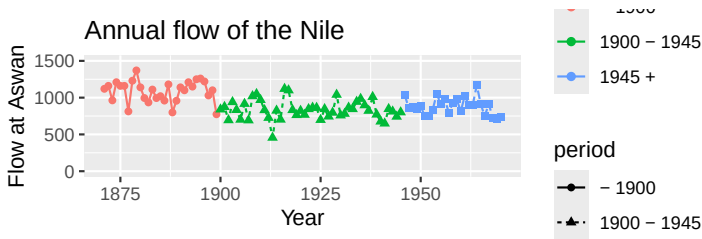
```
pl + theme(legend.position = "bottom")
```



```
pl + facet_wrap(~ period, scales = "free_x") +  
  theme(legend.position = "none")
```



```
pl + labs(title = "Annual flow of the Nile",  
          x = "Year", y = "Flow at Aswan",  
          colour = "Period") +  
scale_y_continuous(limits = c(0, 1500))
```



```
ggsave("nile.pdf", plot = pl, width = 6, height = 4)  
ggsave("nile.png", plot = pl, dpi = 300)
```

A `ggplot` object is just a list – you can build it in a function, return it, put it in a package and document it (as `irisCourse` does).

- A programming paradigm
- Mutable states
- The key abstraction is **an object**
- R is *not* purely object oriented

class the template and generator of objects, e.g. **Account**

constructor sets all fields to default values (organises the memory allocated for the object)

instance / **object** a concrete **Account**, e.g. Måns' account

fields the data an object carries

methods the operations an object offers, e.g. **withdraw()**, **insert()**

interface the collection of “services” the class offers – the public interface has to be exported in NAMESPACE

Class Account

Fields

- `currency`: class variable (shared by all accounts)
- `current_amount`: object variable
- `no_withdraws`: object variable

Methods

- `insert()`
- `withdraw()`

A `SavingsAccount` **inherits** the methods of `Account`: it *is* an `Account`, plus some extra.

Inheritance is possible from multiple classes – your class can extend more than one class.

That immediately raises a question we come back to: if two parents both define a method, *which one runs?*

| | S3 | S4 | RC |
|---------|--------------|--------------|-------------------|
| | Simple | More formal | Latest (R 2.12) |
| Methods | to functions | to functions | to objects |
| Fields | attributes | @ slots | fields |
| Copy | on modify | on modify | no copy-on-modify |
| Access | \$, attr | @ | \$ |

- “Not really objects, more of a naming convention”
- Polymorphic functions, `function_name.class`: for `print`, R calls `print.lm`, `print.anova`, etc. and if nothing is found `print.default`

```
x <- 1:100  
class(x) <- "my_numeric"  
class(x)
```

```
## [1] "my_numeric"
```

S3: methods belong to functions

The generic dispatches, the method does the work.

```
# S3 classes have their own implementation  
# of a function called f  
f <- function(x) UseMethod("f")  
f.my_numeric <- function(x) cat("f() for my_numeric\n")  
f.default <- function(x) cat("f() for anything else\n")  
f(x)
```

```
## f() for my_numeric
```

```
f(1:10)
```

```
## f() for anything else
```

S3: `print()` is a generic function

```
print.my_numeric <- function(x, ...) {  
  cat("This is my numeric vector.\n")  
}  
x           # calls print(x), which dispatches
```

```
## This is my numeric vector.
```

See 732A94_AdvancedRHT2026_Lecture04_S3code.R

Using `.` in the names of your own functions and objects is discouraged.

`t.test()`: is that the `t` method for `test` objects? It is not – but R cannot tell the difference.

S3: constructor, validator, helper

```
new_temperature <- function(x, unit) {  
  stopifnot(is.numeric(x), unit %in% c("C", "F"))  
  structure(x, class = "temperature", unit = unit)  
}  
print.temperature <- function(x, ...)  
  cat(unclass(x), attr(x, "unit"), "\n")  
new_temperature(21.5, "C")
```

```
## 21.5 C
```

```
new_temperature("cold", "C")
```

```
## Error in `new_temperature()`:  
## ! is.numeric(x) is not TRUE
```

S4: classes with slots

```
setClass("Person",  
        slots = list(name = "character",  
                      age = "numeric",  
                      salary = "numeric"))  
# inheriting class, can inherit from several  
setClass("Employee", slots = list(boss = "Person"),  
        contains = "Person")  
alice <- new("Person", name = "Alice", age = 40, salary = 100)  
alice@age
```

```
## [1] 40
```

```
bob <- new("Employee", name = "Bob", age = 25,  
          salary = 100, boss = alice)
```

See 732A94_AdvancedRHT2026_Lecture04_S4code.R

S4: methods – create a generic, then instances

```
setGeneric("salary_change", function(p, i) {
  standardGeneric("salary_change")
})
setMethod("salary_change",
  signature(p = "Person", i = "numeric"),
  function(p, i) p@salary + i)
```

```
setMethod("salary_change",
  signature(p = "Employee", i = "numeric"),
  function(p, i) {
    ## method from the parent (contained) class
    nsal <- callNextMethod()
    if (nsal > p@boss@salary) nsal <- p@salary
    nsal
  })
```

S4: copy-on-modify

```
salary_change(bob, 5)
```

```
## [1] 100
```

```
alice@salary <- salary_change(alice, 10)  
salary_change(bob, 5)  # bob's boss is the OLD alice
```

```
## [1] 100
```

```
bob@boss <- alice  
salary_change(bob, 5)
```

```
## [1] 105
```

bob@boss holds a *copy* of alice – changing alice does not change it.

S4: `callNextMethod()` and ambiguity

With multiple inheritance the order in `contains` decides which parent `callNextMethod()` reaches:

```
setClass("Employee", contains = c("Person", "Computer"))  
setClass("Employee2", contains = c("Computer", "Person"))
```

See

732A94_AdvancedRHT2026_Lecture04_S4code_CallNextMethodAmbiguities.R

S4: printing

You *can* define `print.Person()` in S3 style.

BUT: define the method `show` for the class instead – that gives appropriate default displaying, also when `print()` is not called.

```
setMethod("show", "Person", function(object) {  
  cat(paste0(object@name, ": ", object@age, "y/o"))  
  cat("\n")  
})  
alice
```

```
## Alice: 40y/o
```

S4: validity

S4 can check the object for you, every time one is created:

```
setValidity("Person", function(object) {  
  if (object@age < 0) "age must be non-negative" else TRUE  
})
```

```
new("Person", name = "Nobody", age = -1, salary = 0)
```

```
## Error in `validObject()`:
```

```
## ! invalid class "Person" object: age must be non-negative
```

```
Account <- setRefClass("Account",  
  fields = list(balance = "numeric"),  
  methods = list(  
    withdraw = function(x) {  
      balance <<- balance - x  
    },  
    deposit = function(x) {  
      balance <<- balance + x  
    }  
  )  
)
```

Note the <<-: methods modify the object's own field.

See 732A94_AdvancedRHT2026_Lecture04_RCcode.R

RC: objects are mutable

```
a <- Account$new(balance = 100)
b <- a                                # b is NOT a copy
a$balance <- 0; b$balance             # ... so b changed too
```

```
## [1] 0
```

```
d <- a$copy()                        # all RC objects have copy()
a$balance <- 100; c(d$balance, a$balance)
```

```
## [1] 0 100
```

This is the big difference from S3 and S4.

```
Account$methods(show = function() {  
  cat(paste0("Account balance: ", .self$balance, "\n"))  
})  
Account$methods(initialize = function(balance = 0) {  
  .self$balance <- balance  
  cat("Account created!\n")  
})  
e <- Account(300)
```

```
## Account created!
```

```
e
```

```
## Account balance: 300
```

Checking class: use `inherits()`, NOT `class()`

```
x <- 1; class(x) <- c("a", "b", "c")  
class(x)
```

```
## [1] "a" "b" "c"
```

```
class(x) == "a"
```

```
## [1] TRUE FALSE FALSE
```

```
inherits(x, "a")
```

```
## [1] TRUE
```

With multiple classes `class(x) == "a"` returns a *vector* – using it in `if()` is an error in modern R.

class() vs typeof()

```
typeof(x)           # internal R storage type
```

```
## [1] "double"
```

```
y <- 1L; typeof(y)
```

```
## [1] "integer"
```

```
is.numeric(y); is.double(y)
```

```
## [1] TRUE
```

```
## [1] FALSE
```

`class()` is about the OO system, `typeof()` is about how R stores the value in memory.

S3: `S3method(generic, class)`, e.g. `S3method(print, my_numeric)`

S4: put methods in DESCRIPTION Depends:

- `exportClasses(class)`: makes the class publicly available
- `export(class)`: makes the *generator* publicly available
- `exportMethods(method)`: “If a package defines methods for generic functions, those methods should be exported if any of the classes involved are exported”, e.g. `plot()`

RC: same as S4, but typically impossible to extend outside your package.

S4

- `importClassesFrom(package, ...)`
- `importMethodsFrom(package, ...)`

With roxygen2 you write `@export` and `@importFrom` above the code and let `devtools::document()` write NAMESPACE.

In older versions you would use `@exportClass` and `@exportMethod` in general not needed anymore.R

- **R6** “similar to reference classes, except there is no need to separate fields and methods, and you cannot specify the types of the fields” – the most used alternative in practice
- **S7** the newer attempt at unifying S3 and S4; developed by the R Consortium
- **proto** “neat concept (prototypes, not classes), but tricky in practice”
- **R.oo**

<https://stackoverflow.com/questions/9521651/r-and-object-oriented-programming>

Which one should I use?

- **S3** for almost everything – it is what base R and the tidyverse use, it is light and it is what your users expect
- **S4** when you need formal validity checking, multiple dispatch or you work in Bioconductor
- **RC / R6** when you genuinely need mutable state – a connection, a counter, a simulation object

For the labs you will use S3 and RC.

ALWAYS CHECK INPUT!

Questions?

See you next time!